

DIFFERENCE-AWARE RETRIEVAL POLICIES FOR IMITATION LEARNING

Anonymous authors
Anonymous submission

ABSTRACT

Behavior cloning (BC) is attractive because it learns directly from expert demonstrations, but its parametric state-to-action map often becomes brittle under rollout-induced covariate shift. This paper contributes **Difference-Aware Retrieval Policies for Imitation Learning (DARP)**, a semi-parametric alternative that keeps the original BC data available at inference and makes local neighborhood structure part of the policy itself. Instead of predicting actions only from a query state, DARP retrieves the k -nearest expert states, conditions on their actions and relative difference vectors to the query, and aggregates the resulting neighbor-conditioned action candidates. This design contributes an assumption-preserving way to reduce BC variance: it uses the same demonstrations and standard imitation objective as BC, without online expert feedback, new data collection, or task-specific knowledge. We connect the architecture to implicit Laplacian smoothing and show empirically that the resulting contribution is practical across continuous control, robotic manipulation, and visual-feature imitation, improving over standard BC by 15–46% across the tested settings. Code and demos will be released upon acceptance.

1 INTRODUCTION

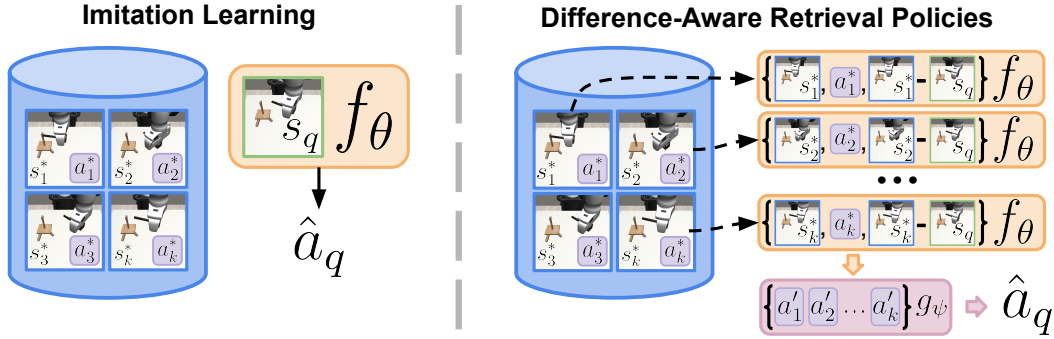


Figure 1: **Contribution overview of DARP:** Standard BC (left) compresses the expert dataset into a parametric policy and discards the data at inference. DARP (right) keeps the dataset active: for each query state s_q , it retrieves a local neighborhood of expert state-action pairs and predicts through difference vectors between the query and retrieved neighbors.

Imitation learning via behavior cloning (BC) (Pomerleau, 1991) has enabled robots to acquire complex, dexterous behaviors from expert demonstrations (Zhao et al., 2023; Chi et al., 2024; Black et al., 2024; Chung et al., 2014). The central limitation we target is that BC’s simplicity comes with a fragile deployment-time interface, especially for long-horizon tasks (Ross et al., 2011): once a learned policy makes small mistakes, the induced trajectory can leave the demonstrated state distribution (Spencer et al., 2021; Ross et al., 2011). In these out-of-distribution regions, the same parametric map that fits expert states well can produce unstable, high-variance actions, making long-horizon rollouts fail even when one-step supervised accuracy is strong.

Many methods address this compounding-error problem, but they often change the learning setting by adding simulators, interactive experts, sub-optimal data, or task-specific structure (Ross et al.,

2011; Venkatraman et al., 2015; Ke et al., 2024b; Levine et al., 2020). Our contribution is deliberately framed within the stricter BC regime: we ask whether the original expert state–action pairs alone can be used more effectively, without extra supervision or feedback. This leads to the paper’s guiding question: *can a policy reduce BC variance at deployment by reusing the demonstration dataset as local context?*

From a statistical perspective, standard BC controls supervised risk on expert states but does not control how the learned function varies between or near those states. This missing control matters most in low-density regions that appear during closed-loop rollouts, where overparameterized policies can oscillate sharply. Smoothness offers a natural remedy: nearby states should induce compatible actions, reducing spurious high-frequency variation and improving rollout stability (Kobayashi, 2022; Asadi et al., 2018; Ke et al., 2024a; Chen et al., 2024). Existing smoothness mechanisms, discussed further in Section 4, can be useful but introduce tradeoffs: augmentation may not enforce consistency, global priors can blur distinct behaviors, temporal penalties act only along trajectories, and explicit graph regularizers add extra smoothness hyperparameters.

A second ingredient is locality. Global supervised policies (Black et al., 2024; Zhao et al., 2023; Chi et al., 2024) must compress all demonstrations into one parametric function, which can be brittle under distribution shift. Local methods (Pari et al., 2022; Mansimov & Cho, 2018; Salzberg & Aha, 1994) instead consult nearby examples and adapt predictions to the observed training distribution. This local grounding is valuable, but purely non-parametric averaging depends strongly on the distance metric, can blur multimodal actions, and often treats neighbors only through absolute states rather than through how each neighbor differs from the query.

We introduce **Difference-Aware Retrieval Policies for Imitation Learning (DARP)**, a semi-parametric policy class that makes these two ideas explicit: retrieve locally, then predict through differences. As illustrated in Figure 1, at inference, DARP retrieves k expert neighbors for the current query state and forms tuples containing each neighbor state, its expert action, and the relative vector from the query to that neighbor. A learned predictor produces a candidate action from each tuple, and a permutation-invariant aggregator combines the candidates into the executed action. This architecture grounds predictions in observed data while using a parametric model to correct local bias and represent richer action structure.

The paper makes three supported contributions. **First**, we formulate difference-aware retrieval as an assumption-preserving replacement for direct state-to-action BC: DARP uses the same expert dataset and standard imitation objective, but exposes local neighborhood structure at both training and inference. **Second**, we show why this architecture should reduce variance by connecting neighbor aggregation to Laplacian smoothing on the expert-state k -NN graph, giving a parameter-free analogue of explicit manifold regularization. **Third**, we validate the design empirically across MuJoCo, Robosuite, RoboCasa, real-world manipulation, visual-feature imitation, and multimodal action modeling, with ablations isolating the importance of difference vectors, meaningful retrieval, and permutation-invariant aggregation.

Together, these contributions show that DARP improves behavior cloning without changing its data assumptions. The theoretical analysis formalizes the smoothing view, while the experiments show 15–46% gains over standard BC across the tested settings and demonstrate compatibility with feedforward networks, set transformers, Gaussian mixture models, diffusion-policy backbones, and high-dimensional visual representations. The result is a general retrieval-based imitation architecture whose value is visible both as a regularization mechanism and as a practical policy design.

2 DIFFERENCE-AWARE RETRIEVAL POLICIES FOR IMITATION LEARNING

This section turns the paper’s main contribution into a concrete method. We first separate the desired effect—local consistency on the expert-data manifold—from the practical architecture that implements it. This structure makes clear what DARP contributes over BC: it keeps the standard imitation-learning data and objective, but changes the policy parameterization so that retrieval and neighbor differences provide an implicit smoothness bias. Readers focused primarily on implementation can skip to Section 2.5 for a self-contained algorithmic summary before the experiments in Section 3.

The development proceeds in four steps. Section 2.1 fixes the BC setting and notation. Section 2.2 introduces an explicit neighbor-manifold regularizer to make the target regularization effect visible. Section 2.3 then shows that the same kind of Laplacian smoothing can be built into the architecture rather than added to the loss. Finally, Section 2.4 instantiates this idea as DARP, a difference-aware retrieval policy compatible with modern neural networks and generative action heads.

2.1 PRELIMINARIES: BEHAVIOR CLONING FOR IMITATION LEARNING

We work in the standard imitation-learning setting so that the contribution can be compared directly to BC. The environment is a finite-horizon Markov Decision Process (MDP), $\mathcal{M} = \{\mathcal{S}, \mathcal{A}, P_0\}$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, and P_0 is the initial state distribution. A *policy* maps states to action distributions, $f_\theta : \mathcal{S} \rightarrow \Delta_{\mathcal{A}}$, to optimize task-relevant objectives; when discussing the function class generally, we omit the subscript θ . We assume only expert demonstrations $\mathcal{D}^* = \{(s_j^*, a_j^*)\}$, with s^* and a^* denoting states and actions from the expert dataset. Behavior cloning (Pomerleau, 1991) trains f_θ as a supervised model by solving $\arg \max_\theta \mathbb{E}_{(s^*, a^*) \sim \mathcal{D}^*} [\log(f_\theta(a^* | s^*))]$. Although f_θ may be an expressive generative model (Lipman et al., 2023; Chi et al., 2024), we begin with a Gaussian parameterization to isolate the contribution of retrieval and smoothing¹.

2.2 WARM-UP: NEIGHBOR MANIFOLD REGULARIZED IMITATION LEARNING

The first step is to state explicitly what BC lacks. BC minimizes imitation loss on expert states in $\mathcal{D}^*$, but the objective gives no direct constraint on policy behavior for states that deviate from the expert manifold. During deployment, accumulated errors can therefore move the agent into regions where an overparameterized BC policy may act arbitrarily (Ross & Bagnell, 2010). The contribution of this warm-up is not yet the final algorithm; it is a clean regularized objective that exposes the smoothness property we later build into DARP.

We encode this property as local consistency: nearby expert states should receive compatible policy predictions. Let $\mathcal{N}_k(s^*)$ denote the k -nearest expert neighbors of s^* , and let $w_i(s^*)$ be normalized kernel weights. The following objective combines the standard supervised BC loss $\mathcal{L}_{\text{BC}}$ with a graph-based smoothness penalty $\mathcal{L}_S$, making the desired manifold structure explicit rather than relying only on pointwise labels.

$$\mathcal{L}_{\text{MRIL}}(f) = \underbrace{\mathbb{E}_{(s^*, a^*) \sim \mathcal{D}^*} [\ell(f(s^*), a^*)]}_{\text{supervised risk } (\mathcal{L}_{\text{BC}})} + \underbrace{\lambda \mathbb{E}_{s^* \sim \mathcal{D}^*} \left[\sum_{i \in \mathcal{N}_k(s^*)} w_i(s^*) \|f(s^*) - f(s_i^*)\|_2^2 \right]}_{\text{smoothness regularizer } (\mathcal{L}_S)}, \quad (1)$$

where $\ell(f(s^*), a^*)$ is the supervised imitation loss, $\mathcal{N}_k(s^*)$ are the k -nearest neighbors of s^* from the expert dataset, and $w_i(s^*) \propto K_\Delta\left(\frac{\|s_i^* - s^*\|}{h}\right)$ are normalized weights based on state differences. This construction is important because it turns local consistency into manifold regularization: as detailed in Appendix A.1.1, the penalty is a Laplacian smoothing term on the expert-state graph, and it yields variance, stability, and generalization benefits.

Theorem 1 (Manifold Regularized BC ($\mathcal{L}_{\text{MRIL}}$) improves over vanilla BC ($\mathcal{L}_{\text{BC}}$)). *Let $f^* : \mathcal{S} \rightarrow \mathcal{A}$ be the true, underlying expert policy, assumed to be C^2 -smooth on a compact state space $\mathcal{S}$. Let $f : \mathcal{S} \rightarrow \mathcal{A}$ denote the learned policy estimator. Consider two estimators trained on expert demonstrations:*

1. **Vanilla BC:** a global supervised model minimizing

$$\mathcal{L}_{\text{BC}}(f) = \mathbb{E}_{(s^*, a^*) \sim \mathcal{D}^*} [\ell(f(s^*), a^*)].$$

2. **MRIL:** a neighbor-based estimator minimizing

$$\mathcal{L}_{\text{MRIL}}(f) = \mathcal{L}_{\text{BC}}(f) + \lambda \mathbb{E}_{s^* \sim \mathcal{D}^*} \left[\sum_{i \in \mathcal{N}_k(s^*)} w_i(s^*) \|f(s^*) - f(s_i^*)\|_2^2 \right],$$

where $w_i(s^*)$ are the kernel weights defined above and $\lambda > 0$.

¹We show that this can be relaxed in Section 2.4

Then, under the smoothness assumption on f , the following hold:

- (i) Variance reduction: The Laplacian penalty in MRIL acts as a data-dependent Tikhonov regularizer, yielding smaller estimator variance than vanilla BC.
- (ii) Smoothness guarantee: Minimizers of $\mathcal{L}_{\text{MRIL}}$ satisfy a uniform bound on the local Lipschitz constant of f , whereas vanilla BC admits interpolants with arbitrarily large Lipschitz constants between training states.
- (iii) Policy stability: In a closed-loop rollout, the deviation recursion

$$\Delta_{t+1} \leq L_s \Delta_t + L_a \|f(s_t) - f^*(s_t^*)\|^2$$

accumulates error linearly for vanilla BC, but sublinearly for MRIL, since the smoothness regularizer enforces $\|f(s^*) - f(s^{j*})\| = O(\|s^* - s^{j*}\|)$ for neighbors s^*, s^{j*} .

This suggests that MRIL enjoys strictly better generalization and stability guarantees than BC.

Proof sketch. We defer the detailed proof to Appendix A.1.1. The proof first identifies the smoothness term as a graph Laplacian penalty on the k -nearest-neighbor (k -NN) affinity graph induced by the kernel weights w_i . In the large-sample limit, this penalty converges to the weighted Dirichlet energy (Belkin & Niyogi, 2008; Zhou et al., 2003). Minimizing that energy gives the learned f local Lipschitz control almost everywhere, which improves rollout stability, and the same quadratic form acts as Tikhonov regularization, reducing estimator variance while keeping bias controlled. $\square$

The takeaway for the final method is that local consistency is more than a heuristic. The smoothness penalty suppresses pairwise disagreement between neighbors in a way that respects the data manifold, shrinking the local Lipschitz constant of f along high-density directions. Consequently, small state changes lead to more consistent actions on in-distribution (ID) states and to more structured extrapolation for nearby out-of-distribution (OOD) states.

2.3 IMPLICIT MANIFOLD REGULARIZATION VIA IN-CONTEXT ARCHITECTURES

The explicit objective above clarifies the desired regularization, but it also exposes two limitations that motivate the paper’s architectural contribution. First, MRIL introduces a new hyperparameter λ to trade off supervised accuracy and smoothness. Second, optimizing a modified regularized loss can change the optimization landscape relative to standard BC. We therefore ask whether the same regularizing effect can be obtained by changing the policy architecture while leaving the imitation objective unchanged.

This section answers that question with an implicit manifold-regularized architecture, iMRIL. The key contribution is to move neighborhood aggregation from the objective into the predictor itself. As shown conceptually in Fig. 2, this makes Laplacian-style smoothing a built-in consequence of inference through neighbors, so the policy can be trained with a standard BC objective and no additional λ . Section 2.4 then adds the difference-aware parameterization that turns this principle into DARP.

iMRIL architecture: Instead of learning a standard feedforward predictor $f(s)$ and separately penalizing nonsmooth behavior, iMRIL parameterizes the action predictor $\hat{f}$ as an average of predictions made at retrieved training neighbors. For an expert state s^* , the model learns a per-state predictor f_θ such that the neighborhood-aggregated prediction $\hat{f}(s^*) = \frac{1}{k} \sum_{i \in \mathcal{N}_k(s^*)} f_\theta(s_i^*)$ matches the expert action a^* . The training objective is therefore still an ordinary imitation loss, but the prediction being

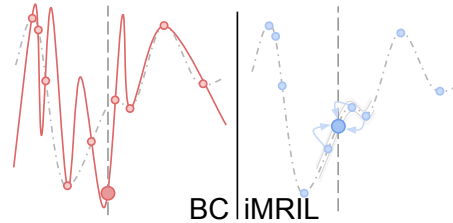


Figure 2: **Architectural contribution: iMRIL builds Laplacian smoothing into prediction.** Neighbor aggregation reduces variance and enforces local consistency, whereas unconstrained BC can fit arbitrarily jagged functions between expert states.

² L_s and L_a are the Lipschitz constants of the environment transition dynamics with respect to state and action, respectively.

supervised is already constrained by neighbor aggregation:

$$\arg \min_{\theta} \mathbb{E}_{(s^*, a^*) \sim \mathcal{D}^*} \left[\left\| \underbrace{\left(\frac{1}{k} \sum_{i \in \mathcal{N}_k(s^*)} f_{\theta}(s_i^*) \right)}_{\hat{f}(s^*)} - a^* \right\|_2 \right] \quad (2)$$

At deployment, iMRIL uses the same interface: retrieve the k -NN of a query state s_q from the training set and compute $\hat{a} = \hat{f}(s_q) = \frac{1}{k} \sum_{i \in \mathcal{N}_k(s_q)} f_{\theta}(s_i^*)$. This establishes the architectural route to smoothing, but it leaves open what information f_{θ} should consume. Section 2.4 shows that this parameterization choice is central to empirical performance and leads directly to DARP.

Intuitively, supervision is applied after aggregation, so individual neighbor predictions cannot vary arbitrarily without affecting the final supervised prediction. This couples nearby predictions and yields the same kind of local consistency identified in Section 2.2. The next result makes that contribution precise by relating iMRIL to Laplacian regularization.

Equivalence between iMRIL and MRIL: We defer the full equivalence proof to Appendix Section A.1, and state the main result here to make the contribution visible: architectural neighbor aggregation implements a parameter-free low-pass filter on the expert-state graph.

Theorem 2 (iMRIL is parameter-free Laplacian regularization for BC (MRIL)). *Consider the symmetric normalized k -NN graph Laplacian L (defined in Section 2.2), with eigenpairs $\{(\mu_j, u_j)\}_{j=1}^n$, where $0 = \mu_1 \leq \mu_2 \leq \dots \leq \mu_n \leq 2$.*

The minimizers of the explicit MRIL objective (Section 2.2) and the implicit iMRIL objective (Section 2.3) have the following closed form expansions

$$f_{\text{MRIL}} = \sum_{j=1}^n \frac{1}{1 + \lambda \mu_j} \langle a^*, u_j \rangle u_j \quad \hat{f}_{\text{iMRIL}} = \sum_{j=1}^n (1 - \mu_j) \langle f, u_j \rangle u_j$$

iMRIL's neighbor aggregation step applies the fixed spectral filter $\phi_{\text{iMRIL}}(\mu) = 1 - \mu$ to the graph Laplacian L , preserving low-frequency modes and suppressing high-frequency modes. The congruence between $\hat{f}_{\text{iMRIL}}$ and f_{MRIL} shows that iMRIL is equivalent to a built-in form of Laplacian smoothing (MRIL) with effective $\lambda \approx 1$ in normalized units. Unlike explicit regularization, this implicit filter requires no additional hyperparameter tuning.

Proof sketch. We defer full details to Appendix Section A.1.2. The explicit regularizer is diagonalized in the k -NN Laplacian eigenbasis, yielding the filter $(1 + \lambda \mu)^{-1}$ on each graph Fourier mode. Neighbor aggregation can be written as $\hat{f} = S f$ with $S = D^{-1} A$, the random-walk matrix, which shares the relevant eigenstructure and applies the fixed filter $1 - \mu$. Both filters preserve smooth, low-frequency variation across the data manifold and damp rapid high-frequency fluctuations between neighbors. Thus iMRIL provides the variance-reduction contribution of Laplacian smoothing without tuning λ . $\square$

This smoothing view clarifies the division of labor in the architecture. Learning remains necessary: the neighbor-conditioned network f_{θ} must predict how expert actions change under local perturbations. Aggregation then reduces variance by smoothing those local proposals across the retrieved neighborhood. In short, the learned predictor corrects local bias, while retrieval aggregation supplies stability.

2.4 DIFFERENCE-AWARE RETRIEVAL POLICIES: A PRACTICAL INSTANTIATION OF iMRIL FOR IMITATION LEARNING

We now instantiate the practical algorithm that carries the theoretical contribution into large-scale imitation learning. Starting from Eq. 2, DARP makes two design choices explicit: the neighbor predictor should be difference-aware, and the final action should be produced by a permutation-invariant aggregation of neighbor-conditioned candidates.

2.4.1 DIFFERENCE-BASED PARAMETERIZATION OF f_θ

Eq. 2 specifies aggregation but not the representation used by f_θ . The practical contribution of DARP is to make each retrieved neighbor informative through its relation to the query. Rather than evaluating f_θ only on s_i^* , DARP gives the predictor the neighbor state s_i^* , the neighbor expert action a_i^* , and the difference vector $\Delta s_i = s_i^* - s_q$. The resulting candidate action is $a'_i = f_\theta(s_i^*, a_i^*, \Delta s_i = s_i^* - s_q)$. This representation lets the model learn how to adapt an observed expert action from a nearby state to the current query state.

Let $\mathcal{N}_k(s_q)$ be the index set of the k -nearest neighbors retrieved according to a distance function $d(s_q, s_i^*)$.³ Given this neighborhood, DARP predicts by aggregating the neighbor-conditioned candidate actions defined above:

$$\hat{a}_q = f_{\text{DARP}}(s_q) = \frac{1}{k} \sum_{i \in \mathcal{N}_k(s_q)} a'_i \quad (3)$$

$$= \frac{1}{k} \sum_{i \in \mathcal{N}_k(s_q)} f_\theta(s_i^*, a_i^*, \Delta s_i = s_i^* - s_q). \quad (4)$$

Training remains deliberately BC-like. For expert query pairs from $\mathcal{D}^*$, we optimize the predictor parameters so that the aggregated DARP prediction matches the demonstrated action:

$$\arg \min_{\theta} \mathbb{E}_{(s_q^*, a_q^*) \sim \mathcal{D}^*} [\|f_{\text{DARP}}(s_q^*) - a_q^*\|^2] \quad (5)$$

This objective is intentionally simple: the method’s contribution lies in the retrieval-conditioned parameterization, not in extra supervision. We optimize f_θ to minimize the discrepancy between the predicted action $\hat{a}_q$ and expert action a_q^* , using standard feedforward or convolutional networks in our experiments. Section 3 shows that the difference vector is not cosmetic; it is a key contributor to performance. At inference, actions are produced by retrieving k -NN and applying the aggregation operation in Eq. 3.

2.4.2 GOING BEYOND LINEAR AGGREGATION

The averaging operation $\hat{a}_q = \frac{1}{k} \sum_{i \in \mathcal{N}_k(s_q)} a'_i$ is the simplest permutation-invariant aggregator, but DARP is not limited to this choice. More expressive set functions $g_\psi(\{a'_i\}_{i \in \mathcal{N}_k(s_q)})$, such as set transformers (Lee et al., 2019) or DeepSets (Zaheer et al., 2017), can replace the average while preserving permutation invariance. This yields the generalized predictor $\hat{a}_q = g_\psi(\{f_\theta(s_i^*, a_i^*, \Delta s_i = s_i^* - s_q)\}_{i \in \mathcal{N}_k(s_q)})$. Beyond expressivity, a parametric aggregator allows DARP to predict richer action distributions, such as Gaussian mixture models (Pignat & Calinon, 2019) or diffusion models (Chi et al., 2024), instead of the unimodal Gaussian implicit in the L_2 objective of Eq. 5. In that setting, DARP predicts distribution parameters α for $p(a_q; \alpha)$ and trains by maximum likelihood:

$$\arg \max_{\theta} \mathbb{E}_{(s_q^*, a_q^*) \sim \mathcal{D}^*} [\log p(a_q^*; \alpha_\theta(s_q^*))], \text{ where } \alpha_\theta(s_q^*) = g_\psi(\{f_\theta(s_i^*, a_i^*, \Delta s_i = s_i^* - s_q^*)\}_{i \in \mathcal{N}_k(s_q^*)}) \quad (6)$$

At inference for a query state s_q , DARP retrieves neighbors, constructs $\alpha_\theta(s_q) = g_\psi(\{f_\theta(s_i^*, a_i^*, \Delta s_i = s_i^* - s_q)\}_{i \in \mathcal{N}_k(s_q)})$, and samples $\hat{a}_q \sim p(\cdot; \alpha_\theta(s_q))$. Appendix Section A.3 provides detailed training pseudocode.

2.5 SUMMARY

The method contribution can be read as a three-part pipeline. First, local consistency in BC corresponds to manifold regularization and yields variance, smoothness, and stability benefits relative to standard BC (Theorem 1). Second, those benefits can be induced implicitly by building neighborhood aggregation into the policy architecture rather than adding an explicit loss term and tuning λ (Theorem 2). Third, DARP makes the implicit architecture practical: for each expert pair (s_q^*, a_q^*) , it retrieves

³In our work we use the Euclidean distance in a pre-trained embedding space, although other neighborhood functions are also applicable. We refer the reader to Appendix Section A.2.1 for a thorough discussion of design decisions in constructing neighborhood sets via retrieval.

k expert neighbors from $\mathcal{D}^*$, computes difference vectors $\Delta s_i = s_i^* - s_q^*$, maps each tuple $(s_i^*, a_i^*, \Delta s_i)$ through f_θ to produce candidate actions, and aggregates them with a permutation-invariant function g_ψ . The algorithm is trained with the standard imitation objective while making the dataset’s local structure visible to the policy.

3 EXPERIMENTAL EVALUATION

The experiments are organized to make the empirical contribution easy to audit. We evaluate three questions aligned with the method claims: **Q1:** Does DARP consistently improve over standard behavior cloning under the same data assumptions? **Q2:** Does the retrieval-and-difference design remain useful with richer state representations and action distributions? **Q3:** Which architectural components are responsible for the gains? Across these questions, we test continuous control, robotic manipulation, high-dimensional visual features, and settings designed to stress the neighbor-based design.

3.1 BASELINE COMPARISONS AND TASK DESCRIPTIONS

MuJoCo Tasks: The MuJoCo (Todorov et al., 2012; Fu et al., 2020) experiments test whether the core contribution transfers to continuous-control locomotion. The tasks require different legged embodiments to move forward on a flat plane: Hopper (single-legged hopping robot), Walker (bipedal humanoid), Ant (quadruped), and HalfCheetah (biped).

Robosuite Tasks: The Robosuite (Zhu et al., 2020) experiments test the same contribution in simulated manipulation. Each task uses a single robotic arm: Stack requires placing a smaller cube on a larger cube, Threading requires inserting a thin needle-like tool into a small ring, and Square Peg requires placing a square wooden block with a central hole onto a square peg.

RoboCasa Tasks: The RoboCasa (Nasiriany et al., 2024) experiments broaden the manipulation setting to randomized kitchens. A single robotic arm closes an open drawer in Drawer, closes an open microwave door in Door, and turns a stove knob off in Stove.

Baseline Comparisons: We compare DARP against baselines that isolate different alternatives to the proposed contribution: (1) **R&P (Sridhar et al., 2025):** execute the action from the nearest neighbor directly; (2) **LWR (Pari et al., 2022):** aggregate neighbors with locally weighted regression; (3) **BC:** standard parametric behavior cloning; (4) **REGENT (Sridhar et al., 2025):** a transformer-based in-context method conditioned on retrieved neighbors; and (5) **MRIL:** the explicitly smoothed objective from Section 2.2.

3.2 CAN DARP CONSISTENTLY OUTPERFORM STANDARD BEHAVIOR CLONING? (Q1)

Q1 tests the main empirical claim in the cleanest setting: low-dimensional state representations, where state distances are meaningful and retrieval can be directly interpreted. We evaluate MuJoCo locomotion and Robosuite/RoboCasa manipulation, with manipulation data generated using MimicGen (Mandlekar et al., 2023). In these experiments, g is the simplest permutation-invariant aggregator, the average over neighbor action predictions a' .

Method	Hopper	Ant	Walker	HalfCheetah
R&P (Sridhar et al., 2025)	711.82 $\pm$ 85.63	-305.97 $\pm$ 76.42	419.18 $\pm$ 50.21	-178.64 $\pm$ 29.75
LWR (Pari et al., 2022)	1703.78 $\pm$ 245.95	846.59 $\pm$ 216.06	1484.91 $\pm$ 356.54	1945.82 $\pm$ 567.26
BC	2313.65 $\pm$ 203.75	2376.20 $\pm$ 339.43	2658.40 $\pm$ 274.08	1063.23 $\pm$ 371.08
REGENT (Sridhar et al., 2025)	1819.39 $\pm$ 186.24	-302.10 $\pm$ 146.67	507.01 $\pm$ 76.10	169.85 $\pm$ 63.10
MRIL	2793.63 $\pm$ 156.41	3869.08 $\pm$ 241.00	4370.96 $\pm$ 168.13	701.58 $\pm$ 195.08
DARP	3545.57 $\pm$ 3.54	4383.28 $\pm$ 266.37	4894.01 $\pm$ 75.12	5515.41 $\pm$ 841.33
DARP Set Transformer	2965.86 $\pm$ 103.08	4063.79 $\pm$ 218.80	4752.42 $\pm$ 109.23	3417.85 $\pm$ 764.57

Table 1: **Q1 result: DARP is the strongest MuJoCo method across the tested low-dimensional tasks.** Scores are averaged across 100 independent trials with 95% confidence intervals. Parametric policies use Multi-Layer Perceptrons; diffusion-policy results are reported in Section 8.

Table 1 and Table 2 support the central empirical contribution: DARP improves over standard BC across all tested environments. The gains are 15–25 percentage points in robotic manipulation and

substantial score increases in locomotion. The comparison also clarifies why the proposed semi-parametric design matters: purely non-parametric retrieval methods (R&P and LWR) are weak on these tasks, and MRIL usually improves over vanilla BC, but the best scores in this suite come from the full DARP architecture.

The next question is whether the same contribution remains visible when the state representation becomes more complex. We therefore extend the evaluation from compact states to visual-feature inputs.



Figure 3: RoboCasa “Drawer-Close” task used to test whether DARP improves BC in randomized kitchen manipulation.



Figure 4: Robosuite “Threading” task used to evaluate retrieval-conditioned manipulation with precise insertion.

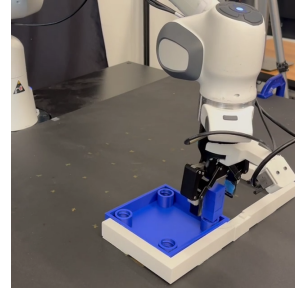


Figure 5: Real-world FurnitureBench square-table assembly task, where the robot picks up a table leg and screws it into the tabletop.

3.3 CAN DARP HANDLE MORE COMPLEX STATE REPRESENTATION AND ACTION DISTRIBUTIONS? (Q2)

High-Dimensional Visual Input Representations. Q2 first asks whether the contribution survives beyond low-dimensional state vectors. We evaluate DARP on sim-

Method	Stack	Thrd.	Peg
BC	44	38	17
DARP	75	76	52

Table 3: **Q2 visual-feature result.** Success rates (%) on vision-based Robosuite tasks show that DARP outperforms BC in all three tasks.

Method	RoboCasa			Robosuite			Real
	Drawer	Door	Stove	Stack	Thrd.	Peg	Sq. Table
BC	54	29	28	47	37	46	44
DARP	85	45	43	72	63	62	92

Table 2: **Q1 result across manipulation domains: DARP improves over BC in simulation and the real world.** Scores are success percentages (RoboCasa and Robosuite: 100 trials; Real: 50 trials). RoboCasa and Robosuite policies use Multi-Layer Perceptrons; the real-world policy uses diffusion policy, as discussed in Section 8.

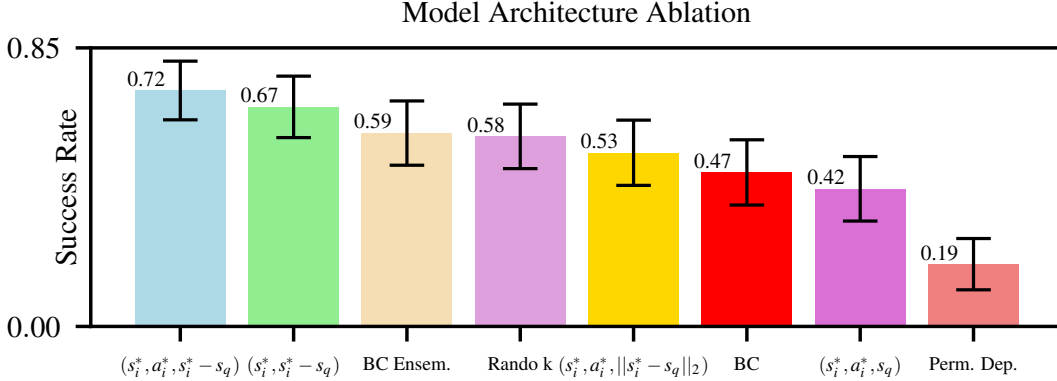


Figure 6: **Q3 ablation: difference vectors and permutation invariance are central to DARP’s gains.** Modifying the core architecture shows that performance depends most strongly on the distance-vector tuple $(s_i^*, a_i^*, s_i^* - s_q)$ and on permutation-invariant aggregation. Success rate is averaged across 100 Robosuite Stack trials with 95% confidence intervals.

ulated robotic manipulation using R3M image embeddings (Nair et al., 2022). Table 3 shows that DARP not only outperforms standard BC with these high-dimensional visual features, but has an average improvement of approximately 35%, larger than the approximately 22% average improvement on low-dimensional Robosuite states. This result supports the claim that difference-aware retrieval can scale to complex learned representations rather than only hand-designed state spaces.

Multi-modal Action Distributions. Q2 also tests whether DARP can be paired with richer action heads. On the multimodal Push-T environment, DARP performs over 20% better than behavior cloning. Appendix A.2.2 reports the details, using the extension from Section 2.4 that predicts parameters of a multimodal action distribution.

3.4 HOW DO DIFFERENT ARCHITECTURAL COMPONENTS CONTRIBUTE TO DARP’S PERFORMANCE GAINS? (Q3)

Ablation Study: Q3 examines whether the gains come from the specific contribution structure of DARP rather than from retrieval alone. We ablate eight variants: (1) standard DARP; (2) DARP without neighbor actions; (3) an ensemble of 10 BC agents; (4) DARP with random neighbors instead of metric-based retrieval; (5) DARP using the L_2 norm of the distance vector; (6) the BC baseline using only the query state s_q ; (7) DARP using the query state instead of the difference vector between query and neighbor states; and (8) DARP with a permutation-dependent, rather than permutation-invariant, aggregator for combining the a' predictions. Figure 6 reports this systematic ablation.

The ablation study isolates the main design contribution. Difference vectors and permutation-invariant aggregation are crucial, while neighbor actions provide a smaller but still relevant contribution. Random neighbor selection performs poorly, confirming that DARP relies on meaningful local

neighborhoods rather than simply increasing model input size. Permutation-dependent aggregation also degrades performance, supporting the use of a set-structured aggregator g .

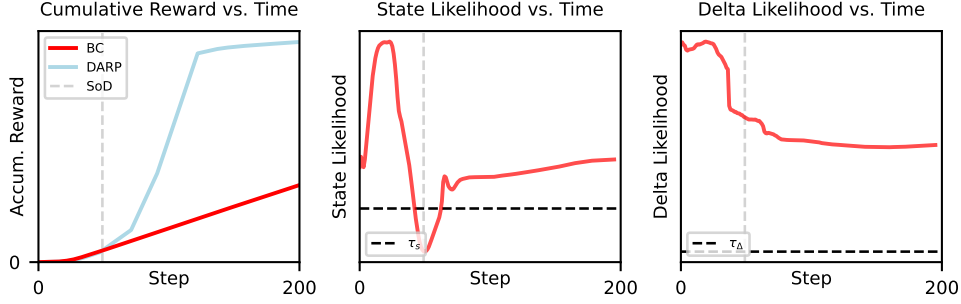


Figure 7: Divergence analysis: DARP succeeds after BC leaves the demonstrated state distribution. Cumulative rewards for BC and DARP on Robosuited Stack are initially identical, then diverge when BC fails and DARP succeeds. The dashed line marks the step of divergence (“SoD”); at that step, state likelihood is $< \tau_s$ (OOD), while delta likelihood is $> \tau_\Delta$ (in distribution).

Divergence Analysis: To connect the empirical gains back to the method’s motivation, we analyze rollouts where BC fails but DARP succeeds. We define the “step of divergence” as the first point at which DARP and BC begin receiving substantially different rewards. We define τ_s and τ_Δ as the 1st percentile of training-set likelihoods, so 1% of training deltas are less likely than τ_Δ . Across six rollouts from two tasks (Robosuited Stack and MuJoCo Hopper), the query state at the SoD has state likelihood $< \tau_s$ but delta likelihood $> \tau_\Delta$. This supports the paper’s central interpretation: DARP can remain effective when the absolute state is slightly OOD because the difference vectors to retrieved neighbors remain in distribution. Figure 7 visualizes reward drift, SoDs, and likelihoods for one task, and Appendix A.2.4 provides additional robustness experiments.

4 RELATED WORK

Non-Parametric Imitation Learning Methods: The closest prior direction is non-parametric IL, which also uses local structure at prediction time. VINN Pari et al. (2022) studies locally weighted regression for imitation and shows strong results in image embedding spaces, while MiDiGaP von Hartz et al. (2025) uses mixtures of Gaussian processes to model multimodal trajectories and generalize quickly. SEABO Lyu et al. (2024) uses retrieval for offline RL by assigning rewards based on proximity to neighbors. FlowRetrieval Lin et al. (2024), STRAP Memmel et al. (2025), and Behavior Retrieval Du et al. (2023) retrieve from large unlabeled datasets and use finetuning or test-time training to improve generalization. The contribution of DARP is different: it parameterizes each retrieved example as $(s_i^*, a_i^*, s_i^* - s_q)$ and trains a semi-parametric policy, rather than relying only on non-parametric aggregation or test-time adaptation. This is what lets DARP combine local variance reduction with the representational flexibility of parametric policies.

Smoothness-Constrained Policy Learning: A second relevant thread explicitly constrains policy smoothness to improve stability and robustness. L2C2 Kobayashi (2022) studies model-free RL with local Lipschitz continuity constraints, improving smoothness and noise robustness without removing expressivity, and Asadi et al. (2018) use related Lipschitz constraints for model-based RL models. CCIL Ke et al. (2024a) extends this perspective to imitation learning by generating synthetic corrective labels with a Lipschitz-constrained dynamics model, and related smoothness ideas have been scaled to humanoid controllers Chen et al. (2024). DARP contributes an architectural route to the same goal: it induces local consistency through retrieval and aggregation while preserving the standard imitation-learning objective.

In-Context Learning Methods: Recent in-context imitation methods also use retrieved data as part of the policy input. REGENT Sridhar et al. (2025) studies retrieval-augmented generalization by conditioning a causal transformer on retrieved states, actions, and rewards, while DPT Lee et al. (2023) uses supervised transformer pretraining to predict actions from query states and in-context datasets. Other in-context architectures include ICRT Fu et al. (2024), Instant Policy Vosylus & Johns (2025), and KAT Di Palo & Johns (2024). These methods primarily target rapid adaptation to new tasks or environments. DARP instead focuses on improving performance and stability in

the standard imitation-learning setting by making retrieved neighbor differences the central policy representation.

5 CONCLUSION

This paper contributes **Difference-Aware Retrieval Policies for Imitation Learning (DARP)** (DARP), a semi-parametric alternative to direct state-to-action behavior cloning. The method keeps the BC data assumptions unchanged, but makes the expert dataset active at inference by retrieving neighbors, representing each neighbor through its action and relative difference to the query, and aggregating neighbor-conditioned action candidates. The analysis identifies this architecture as an implicit form of Laplacian smoothing, explaining how DARP reduces variance without adding an explicit regularization hyperparameter. Empirically, the contribution is supported across continuous control, simulated and real robotic manipulation, visual-feature imitation, and multimodal action modeling: DARP improves over standard BC by 15–46% in the tested scenarios. The ablations further clarify why the method works, showing that difference vectors, meaningful retrieval, and permutation-invariant aggregation are the core architectural ingredients.

6 REPRODUCIBILITY STATEMENT

To make the reported contribution reproducible, we will release supplementary source code, policy and environment configuration files, MuJoCo experiment data, scripts for generating Robosuite expert demonstrations with MimicGen, and modality transformation code upon acceptance. On NVIDIA L40 and L40s GPUs, these materials reproduce the reported results except for transformer-based methods (REGENT and Set Transformer), which are non-deterministic and may differ slightly from the numbers in the paper.

REFERENCES

- Kavosh Asadi, Dipendra Misra, and Michael Littman. Lipschitz continuity in model-based reinforcement learning. In *International Conference on Machine Learning*. PMLR, PMLR, 2018.
- Mikhail Belkin and Partha Niyogi. Towards a theoretical foundation for laplacian-based manifold methods. *Journal of Computer and System Sciences*, 74(8):1289–1308, 2008.
- Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Lucy Xiaoyang Shi, James Tanner, Quan Vuong, Anna Walling, Haohuan Wang, and Ury Zhilinsky. π_0 : A vision-language-action flow model for general robot control, 2024. URL <https://arxiv.org/abs/2410.24164>.
- Zixuan Chen, Xialin He, Yen-Jen Wang, Qiayuan Liao, Yanjie Ze, Zhongyu Li, S. Shankar Sastry, Jiajun Wu, Koushil Sreenath, Saurabh Gupta, and Xue Bin Peng. Learning smooth humanoid locomotion through lipschitz-constrained policies. *arxiv preprint arXiv:2410.11825*, 2024.
- Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, 2024.
- Fan R. K. Chung. *Spectral Graph Theory*, volume 92 of *CBMS Regional Conference Series in Mathematics*. American Mathematical Society, 1997. doi: 10.1090/cbms/092.
- Michael Jae-Yoon Chung, Maxwell Forbes, Maya Cakmak, and Rajesh PN Rao. Accelerating imitation learning through crowdsourcing. In *2014 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 4777–4784. IEEE, 2014.
- Norman Di Palo and Edward Johns. Keypoint action tokens enable in-context imitation learning in robotics. In *Proceedings of Robotics: Science and Systems (RSS)*, 2024.

-
- Maximilian Du, Suraj Nair, Dorsa Sadigh, and Chelsea Finn. Behavior retrieval: Few-shot imitation learning by querying unlabeled datasets. In *Proceedings of Robotics: Science and Systems (RSS)*, 2023.
- Justin Fu, Aviral Kumar, Ofir Nachum, George Tucker, and Sergey Levine. D4rl: Datasets for deep data-driven reinforcement learning, 2020.
- Letian Fu, Huang Huang, Gaurav Datta, Lawrence Yunliang Chen, William Chung-Ho Panitch, Fangchen Liu, Hui Li, and Ken Goldberg. In-context imitation learning via next-token prediction. *arXiv preprint arXiv:2408.15980*, 2024.
- Liyiming Ke, Yunchu Zhang, Abhay Deshpande, Siddhartha Srinivasa, and Abhishek Gupta. Ccil: Continuity-based data augmentation for corrective imitation learning. In *The Twelfth International Conference on Learning Representations*, 2024a. URL <https://openreview.net/forum?id=LQ6LQ8f4y8>.
- Liyiming Ke, Yunchu Zhang, Abhay Deshpande, Siddhartha S. Srinivasa, and Abhishek Gupta. CCIL: continuity-based data augmentation for corrective imitation learning. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024b. URL <https://openreview.net/forum?id=LQ6LQ8f4y8>.
- Taisuke Kobayashi. L2c2: Locally lipschitz continuous constraint towards stable and smooth reinforcement learning. In *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 4032–4039. IEEE, IEEE, October 2022.
- Jonathan Lee et al. Supervised pretraining can learn in-context reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, pp. 43057–43083, 2023.
- Juho Lee, Yoonho Lee, Jungtaek Kim, Adam Kosior, Seungjin Choi, and Yee Whye Teh. Set transformer: A framework for attention-based permutation-invariant neural networks. In *Proceedings of the 36th International Conference on Machine Learning*, pp. 3744–3753, 2019.
- Sergey Levine, Aviral Kumar, George Tucker, and Justin Fu. Offline reinforcement learning: Tutorial, review, and perspectives on open problems, 2020. URL <https://arxiv.org/abs/2005.01643>.
- Li-Heng Lin, Yuchen Cui, Amber Xie, Tianyu Hua, and Dorsa Sadigh. Flowretrieval: Flow-guided data retrieval for few-shot imitation learning. In Pulkit Agrawal, Oliver Kroemer, and Wolfram Burgard (eds.), *Conference on Robot Learning, 6-9 November 2024, Munich, Germany*, volume 270 of *Proceedings of Machine Learning Research*, pp. 4084–4099. PMLR, 2024. URL <https://proceedings.mlr.press/v270/lin25a.html>.
- Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matthew Le. Flow matching for generative modeling. In *International Conference on Learning Representations (ICLR) 2023*, 2023. URL <https://arxiv.org/abs/2210.02747>. Originally appeared as arXiv preprint arXiv:2210.02747.
- Jiafei Lyu, Xiaoteng Ma, Le Wan, Runze Liu, Xiu Li, , and Zongqing Lu. Seabo: A simple search-based method for offline imitation learning. In *International Conference on Learning Representations*, 2024.
- Ajay Mandlekar, Soroush Nasiriany, Bowen Wen, Ireteyio Akinola, Yashraj Narang, Linxi Fan, Yuke Zhu, and Dieter Fox. Mimicgen: A data generation system for scalable robot learning using human demonstrations. In *7th Annual Conference on Robot Learning*, 2023.
- Elman Mansimov and Kyunghyun Cho. Simple nearest neighbor policy method for continuous control tasks. In *International Conference on Learning Representations (ICLR) 2018*, 2018. Under review as a conference paper at ICLR 2018.
- Marius Memmel, Jacob Berg, Bingqing Chen, Abhishek Gupta, and Jonathan Francis. Strap: Robot sub-trajectory retrieval for augmented policy learning. In *The Thirteenth International Conference on Learning Representations*, 2025.

- Suraj Nair, Aravind Rajeswaran, Vikash Kumar, Chelsea Finn, and Abhinav Gupta. R3M: A universal visual representation for robot manipulation. In Karen Liu, Dana Kulic, and Jeffrey Ichnowski (eds.), *Conference on Robot Learning, CoRL 2022, 14-18 December 2022, Auckland, New Zealand*, volume 205 of *Proceedings of Machine Learning Research*, pp. 892–909. PMLR, 2022. URL <https://proceedings.mlr.press/v205/nair23a.html>.
- Soroush Nasiriany, Abhiram Maddukuri, Lance Zhang, Adeet Parikh, Aaron Lo, Abhishek Joshi, Ajay Mandlekar, and Yuke Zhu. Robocasa: Large-scale simulation of everyday tasks for generalist robots. In *Robotics: Science and Systems (RSS)*, 2024.
- Jyothish Pari, Nur Muhammad Shafullah, Sridhar Pandian Arunachalam, and Lerrel Pinto. The surprising effectiveness of representation learning for visual imitation. In *Robotics: Science and Systems (RSS)*. Robotics: Science and Systems Foundation, June 2022. doi: 10.15607/RSS.2022.XVIII-052.
- Emmanuel Pignat and Sylvain Calinon. Bayesian gaussian mixture model for robotic policy imitation. *IEEE Robotics and Automation Letters*, 2019. URL <https://arxiv.org/pdf/1904.10716.pdf>. Preprint version.
- Dean Pomerleau. Efficient training of artificial neural networks for autonomous navigation. *Neural Comput.*, 3(1):88–97, 1991. doi: 10.1162/NECO.1991.3.1.88. URL <https://doi.org/10.1162/neco.1991.3.1.88>.
- Stéphane Ross and Drew Bagnell. Efficient reductions for imitation learning. In *Proceedings of the thirteenth international conference on artificial intelligence and statistics*, pp. 661–668. JMLR Workshop and Conference Proceedings, 2010.
- Stéphane Ross, Geoffrey J. Gordon, and Drew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In Geoffrey J. Gordon, David B. Dunson, and Miroslav Dudík (eds.), *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics, AISTATS 2011, Fort Lauderdale, USA, April 11-13, 2011*, volume 15 of *JMLR Proceedings*, pp. 627–635. JMLR.org, 2011. URL <http://proceedings.mlr.press/v15/ross11a/ross11a.pdf>.
- Steven L. Salzberg and David W. Aha. Learning to catch: Applying nearest neighbor algorithms to dynamic control tasks. In P. Cheeseman and R. W. Oldford (eds.), *Selecting Models from Data: Artificial Intelligence and Statistics IV*, volume 89 of *Lecture Notes in Statistics*, pp. 321–328. Springer, 1994. doi: 10.1007/978-1-4612-2660-4_33. URL https://link.springer.com/chapter/10.1007/978-1-4612-2660-4_33.
- Jonathan C. Spencer, Sanjiban Choudhury, Arun Venkatraman, Brian D. Ziebart, and J. Andrew Bagnell. Feedback in imitation learning: The three regimes of covariate shift. *CoRR*, abs/2102.02872, 2021. URL <https://arxiv.org/abs/2102.02872>.
- Kaustubh Sridhar, Souradeep Dutta, Dinesh Jayaraman, and Insup Lee. REGENT: A retrieval-augmented generalist agent that can act in-context in new environments. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025. Oral presentation.
- Emanuel Todorov, Tom Erez, and Yuval Tassa. Mujoco: A physics engine for model-based control. In *2012 IEEE/RSJ international conference on intelligent robots and systems*, pp. 5026–5033. IEEE, 2012.
- Arun Venkatraman, Martial Hebert, and J. Andrew Bagnell. Improving multi-step prediction of learned time series models. In Blai Bonet and Sven Koenig (eds.), *Proceedings of the Twenty-Ninth AAAI Conference on Artificial Intelligence, January 25-30, 2015, Austin, Texas, USA*, pp. 3024–3030. AAAI Press, 2015. doi: 10.1609/AAAI.V29I1.9590. URL <https://doi.org/10.1609/aaai.v29i1.9590>.
- Jan Ole von Hartz, Adrian Röfer, Joschka Boedecker, and Abhinav Valada. The unreasonable effectiveness of discrete-time gaussian process mixtures for robot policy learning. *arXiv preprint arXiv:2505.03296*, 2025.

-
- Vitalis Vosylius and Edward Johns. Instant policy: In-context imitation learning via graph diffusion. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2025.
- Manzil Zaheer, Satwik Kottur, Siamak Ravanbakhsh, Barnabas Poczos, Ruslan R Salakhutdinov, and Alexander J Smola. Deep sets. In I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett (eds.), *Advances in Neural Information Processing Systems 30*, pp. 3391–3401. Curran Associates, Inc., 2017. URL <http://papers.nips.cc/paper/6931-deep-sets.pdf>.
- Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In *Proceedings of Robotics: Science and Systems (RSS)*, 2023.
- Dengyong Zhou, Olivier Bousquet, Thomas Navin Lal, Jason Weston, and Bernhard Schölkopf. Learning with local and global consistency. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 16, 2003.
- Yuke Zhu, Josiah Wong, Ajay Mandlekar, Roberto Martín-Martín, Abhishek Joshi, Soroush Nasiriany, Yifeng Zhu, and Kevin Lin. robosuite: A modular simulation framework and benchmark for robot learning. In *arXiv preprint arXiv:2009.12293*, 2020.

A APPENDIX

A.1 LEMMAS AND PROOFS

A.1.1 PROOF OF THEOREM 1

This proof supports the paper’s first theoretical contribution: local consistency in BC can be written as graph-Laplacian smoothing. We begin with a standard result (Chung, 1997; Zhou et al., 2003; Belkin & Niyogi, 2008).

Lemma 1 (Smoothness regularizer as k -NN graph Laplacian penalty). *Let $\{s_1^*, \dots, s_n^*\}$ be the expert states with corresponding predicted actions $f(s_i^*) \in \mathbb{R}^{d_a}$. For each i , let $\mathcal{N}_k(s_i^*)$ denote the indices of the k -nearest neighbors of s_i^* (excluding i). Define asymmetric weights*

$$\tilde{W}_{ij} = \begin{cases} w_j(s_i^*), & \text{if } j \in \mathcal{N}_k(s_i^*), \\ 0, & \text{otherwise,} \end{cases}$$

and construct a symmetric affinity matrix

$$W_{ij} = \frac{1}{2} (\tilde{W}_{ij} + \tilde{W}_{ji}).$$

Let D be the degree matrix with $D_{ii} = \sum_j W_{ij}$, and define the k -NN graph Laplacian $L = D - W$.

Then the local-consistency contribution of the smoothness regularizer can be written as the quadratic form

$$\mathcal{L}_S(f) = \frac{1}{n} \sum_{i=1}^n \sum_{j \in \mathcal{N}_k(s_i^*)} w_j(s_i^*) \|f(s_i^*) - f(s_j^*)\|^2 \propto \text{Tr}(F^\top L F),$$

where $F = [f(s_1^*), f(s_2^*), \dots, f(s_n^*)]^\top \in \mathbb{R}^{n \times d_a}$. Equivalently, in the scalar case,

$$\mathcal{L}_S(f) \propto f^\top L f.$$

Corollary 1 (Continuum limit of smoothness regularizer). *Assume states $\{s_i^*\}_{i=1}^n$ are sampled i.i.d. from a smooth density $p(s^*)$ supported on an m -dimensional C^2 manifold $\mathcal{M} \subset \mathbb{R}^d$. Let W be the symmetrized k -NN affinity matrix constructed from a kernel K_Δ with bandwidth h , and let $L = D - W$ be the graph Laplacian.*

If $n \rightarrow \infty$, $h \rightarrow 0$, and $nh^{m+2} \rightarrow \infty$, then the normalized quadratic form converges to the weighted Dirichlet energy:

$$\frac{1}{n^2 h^{m+2}} \text{Tr}(F^\top L F) \longrightarrow C_K \int_{\mathcal{M}} \|\nabla_{\mathcal{M}} f(s)\|_2^2 p(s)^2 d\text{vol}(s),$$

where $C_K > 0$ is a constant depending only on the kernel K_Δ .

Theorem 1 (Manifold Regularized BC ($\mathcal{L}_{\text{MRIL}}$) improves over vanilla BC ($\mathcal{L}_{\text{BC}}$)). *Let $f^* : \mathcal{S} \rightarrow \mathcal{A}$ be the true, underlying expert policy, assumed to be C^2 -smooth on a compact state space $\mathcal{S}$. Let $f : \mathcal{S} \rightarrow \mathcal{A}$ denote the learned policy estimator. Consider two estimators trained on expert demonstrations:*

1. **Vanilla BC:** a global supervised model minimizing

$$\mathcal{L}_{\text{BC}}(f) = \mathbb{E}_{(s^*, a^*) \sim \mathcal{D}^*} [\ell(f(s^*), a^*)].$$

2. **MRIL:** a neighbor-based estimator minimizing

$$\mathcal{L}_{\text{MRIL}}(f) = \mathcal{L}_{\text{BC}}(f) + \lambda \mathbb{E}_{s^* \sim \mathcal{D}^*} \left[\sum_{i \in \mathcal{N}_k(s^*)} w_i(s^*) \|f(s^*) - f(s_i^*)\|_2^2 \right],$$

where $w_i(s^*)$ are the kernel weights defined above and $\lambda > 0$.

Then, under the smoothness assumption on f , the following hold:

- (i) Variance reduction: The Laplacian penalty in MRIL acts as a data-dependent Tikhonov regularizer, yielding smaller estimator variance than vanilla BC.

(ii) Smoothness guarantee: Minimizers of $\mathcal{L}_{\text{MRIL}}$ satisfy a uniform bound on the local Lipschitz constant of f , whereas vanilla BC admits interpolants with arbitrarily large Lipschitz constants between training states.

(iii) Policy stability: In a closed-loop rollout, the deviation recursion

$$\Delta_{t+1} \leq L_s \Delta_t + L_a \|f(s_t) - f^*(s_t^*)\|^4$$

accumulates error linearly for vanilla BC, but sublinearly for MRIL, since the smoothness regularizer enforces $\|f(s^*) - f(s'^*)\| = O(\|s^* - s'^*\|)$ for neighbors s^*, s'^* .

This suggests that MRIL enjoys strictly better generalization and stability guarantees than BC.

Proof. We prove the three claimed benefits of the explicit regularizer in turn.

(i) Variance reduction. The Laplacian penalty in $\mathcal{L}_{\text{MRIL}}$ is

$$\sum_{i,j} W_{ij} \|f(s_i^*) - f(s_j^*)\|^2 = 2f^\top Lf,$$

where $L = D - W$ is the graph Laplacian, and W is the k -NN affinity matrix. By Lemma 1, this is exactly the empirical Dirichlet energy of f on the expert-state k -NN graph. Standard results (Zhou et al., 2003; Belkin & Niyogi, 2008) identify this quadratic penalty with Tikhonov regularization in the graph Laplacian norm $\|f\|_L^2 = f^\top Lf$. Thus the contribution of the regularizer is variance control: relative to unregularized BC, it reduces estimator variance while keeping the bias term controlled.

(ii) Smoothness guarantee. Consider the continuum limit (Corollary 1): for i.i.d. samples $\{s_i^*\}$ from density p on a smooth manifold $\mathcal{M}$, the normalized penalty converges to

$$\int_{\mathcal{M}} \|\nabla f(s)\|^2 p(s)^2 d\text{vol}(s).$$

This is the weighted Dirichlet energy of f on $\mathcal{M}$. If this integral is finite, f belongs to the Sobolev space $H^1(\mathcal{M}, p^2)$, and in particular f is locally Lipschitz almost everywhere with

$$\|f(s^*) - f(s'^*)\| \leq C \|s^* - s'^*\| \quad \text{for } p\text{-a.e. neighbor pairs } s^*, s'^*.$$

Therefore minimizers of $\mathcal{L}_{\text{MRIL}}$ have uniformly bounded local Lipschitz constants along high-density regions of the state space. This is the smoothness contribution that standard BC lacks: a vanilla BC minimizer can use any oscillatory interpolant that matches the training data, so arbitrarily large Lipschitz constants remain possible between observed states.

(iii) Policy stability. Let $\Delta_t = \|s_t - s_t^*\|$ denote the deviation at time t . For Lipschitz dynamics T ,

$$\Delta_{t+1} \leq L_s \Delta_t + L_a \|f(s_t) - f^*(s_t^*)\|.$$

Decompose the action error:

$$\|f(s_t) - f^*(s_t^*)\| \leq \|f(s_t) - f^*(s_t)\| + \|f^*(s_t) - f^*(s_t^*)\|.$$

For vanilla BC, the first term $\|f(s_t) - f^*(s_t)\|$ is controlled only on the empirical distribution $P_{\mathcal{S}}$; off-distribution, it may be $O(1)$ regardless of Δ_t . The second term satisfies $\|f^*(s_t) - f^*(s_t^*)\| = O(\Delta_t)$ by smoothness of f^* . Hence the BC deviation recursion can take the form

$$\Delta_{t+1} \leq L_s \Delta_t + L_a (O(1) + O(\Delta_t)),$$

which accumulates linearly in t .

For MRIL, the Laplacian penalty contributes the missing local control by enforcing

$$\|f(s^*) - f(s'^*)\| \leq C \|s^* - s'^*\| \quad \text{for neighbor pairs } (s^*, s'^*),$$

as shown in part (ii). Thus $\|f(s_t) - f^*(s_t)\| = O(h^2)$ (where h is the radius of the kernel bandwidth around the point s_t) by local-linear regression error bounds, and $\|f^*(s_t) - f^*(s_t^*)\| = O(\Delta_t)$ by smoothness of f^* . Combining these,

$$\Delta_{t+1} \leq L_s \Delta_t + L_a (O(h^2) + O(\Delta_t)).$$

Because the smoothness constraint reduces the constant multiplying Δ_t , cumulative error grows strictly more slowly than under vanilla BC. In particular, error growth is sublinear in the rollout horizon when h is small, whereas vanilla BC can accumulate error linearly.

⁴ L_s and L_a are the Lipschitz constants of the environment transition dynamics with respect to state and action, respectively.

Conclusion. Claims (i)–(iii) establish the theoretical contribution of MRIL: lower variance, uniform smoothness control, and sublinear rollout error accumulation relative to vanilla BC. $\square$

Kernel choice. For the IC smoothness regularizer, the implementation uses a Gaussian kernel

$$w_i(s^*) \propto \exp\left(-\frac{\|s^* - s_i^*\|^2}{2h^2}\right), \quad \sum_{i \in \mathcal{N}_k(s^*)} w_i(s^*) = 1,$$

with bandwidth h set to the median distance to the k -th nearest neighbor across the dataset. This standard manifold-regularization choice (Belkin & Niyogi, 2008; Zhou et al., 2003) preserves the proof requirement that the graph Laplacian penalty converges to the Dirichlet energy in the continuum limit. Empirically, this default was stable across tasks, and alternative kernels such as uniform k -NN or exponential decay produced qualitatively similar results.

A.1.2 PROOF OF THEOREM 2

This proof supports the architectural contribution by showing that explicit Laplacian regularization and neighbor aggregation have aligned spectral forms.

Lemma 2 (Spectral form of explicit Laplacian regularization). *Let L be the symmetric normalized graph Laplacian with eigenpairs $\{(\mu_j, u_j)\}_{j=1}^n$, where $0 = \mu_1 \leq \mu_2 \leq \dots \leq \mu_n \leq 2$. The minimizer of the penalized objective*

$$\mathcal{L}_\lambda(f) = \|f - a^*\|^2 + \lambda f^\top L f$$

has the closed-form expansion

$$f_\lambda = \sum_{j=1}^n \frac{1}{1 + \lambda \mu_j} \langle a^*, u_j \rangle u_j.$$

Thus the explicit method’s tunable contribution is the spectral filter $\phi_\lambda(\mu) = (1 + \lambda \mu)^{-1}$ applied to each Laplacian mode.

Proof. Diagonalize $L = U \Lambda U^\top$ with $U = [u_1, \dots, u_n]$ orthogonal and $\Lambda = \text{diag}(\mu_1, \dots, \mu_n)$. Write $f = U c$, $a^* = U b$ in this basis. The objective becomes

$$\|U c - U b\|^2 + \lambda c^\top \Lambda c = \sum_{j=1}^n (c_j - b_j)^2 + \lambda \mu_j c_j^2.$$

Minimizing each term yields $c_j = \frac{1}{1 + \lambda \mu_j} b_j$. Transforming back gives the stated expansion, making the explicit low-pass filter visible. $\square$

Lemma 3 (Spectral form of neighbor aggregation). *Let $S = D^{-1}A$ be the random-walk matrix of the k -NN graph, with adjacency A and degree D . For any prediction vector f , the neighbor-averaged prediction is $\hat{f} = S f$. In the Laplacian eigenbasis, this corresponds to the spectral filter*

$$\hat{f} = \sum_{j=1}^n (1 - \mu_j) \langle f, u_j \rangle u_j,$$

i.e. $\phi_{\text{DARP}}(\mu) = 1 - \mu$.

Proof. By definition, $L = I - D^{-1/2} A D^{-1/2}$ and $S = D^{-1} A = I - L_{\text{rw}}$ where $L_{\text{rw}} = D^{-1} L$ is the random-walk Laplacian. Since L_{rw} and L share the same spectrum up to similarity transform, the eigenbasis $\{u_j\}$ diagonalizes S . Thus for each mode u_j , $S u_j = (1 - \mu_j) u_j$, yielding the claimed spectral filter. $\square$

Theorem 2 (iMRIL is parameter-free Laplacian regularization for BC (MRIL)). *Consider the symmetric normalized k -NN graph Laplacian L (defined in Section 2.2), with eigenpairs $\{(\mu_j, u_j)\}_{j=1}^n$, where $0 = \mu_1 \leq \mu_2 \leq \dots \leq \mu_n \leq 2$.*

The minimizers of the explicit MRIL objective (Section 2.2) and the implicit iMRIL objective (Section 2.3) have the following closed form expansions

$$f_{\text{MRIL}} = \sum_{j=1}^n \frac{1}{1 + \lambda \mu_j} \langle a^*, u_j \rangle u_j \quad \hat{f}_{\text{iMRIL}} = \sum_{j=1}^n (1 - \mu_j) \langle f, u_j \rangle u_j$$

iMRIL’s neighbor aggregation step applies the fixed spectral filter $\phi_{\text{iMRIL}}(\mu) = 1 - \mu$ to the graph Laplacian L , preserving low-frequency modes and suppressing high-frequency modes. The congruence between $\hat{f}_{\text{iMRIL}}$ and f_{MRIL} shows that *iMRIL* is equivalent to a built-in form of Laplacian smoothing (MRIL) with effective $\lambda \approx 1$ in normalized units. Unlike explicit regularization, this implicit filter requires no additional hyperparameter tuning.

Proof. From Lemma 3, the neighbor aggregation operator $S = D^{-1}A$ acts on Laplacian eigenmodes u_j as

$$Su_j = (1 - \mu_j)u_j,$$

where μ_j are the normalized Laplacian eigenvalues. Thus in the graph Fourier basis, neighbor aggregation corresponds to multiplying each mode by the fixed spectral filter $\phi_{\text{DARP}}(\mu) = 1 - \mu$.

On the other hand, Lemma 2 shows that explicit Laplacian regularization with parameter λ yields the spectral filter $\phi_\lambda(\mu) = (1 + \lambda \mu)^{-1}$. Both filters downweight high-frequency modes ($\mu \gg 0$) while preserving low-frequency modes ($\mu \approx 0$). The difference relevant to the method contribution is that $\phi_\lambda(\mu)$ requires tuning λ , whereas $\phi_{\text{DARP}}(\mu)$ is parameter-free.

To see the equivalence, note that for small μ ,

$$\phi_{\text{DARP}}(\mu) = 1 - \mu \approx (1 + \mu)^{-1} = \phi_{\lambda=1}(\mu) \quad \text{up to } O(\mu^2) \text{ terms.}$$

Thus DARP can be interpreted as performing Laplacian smoothing with an effective regularization weight of order $\lambda \approx 1$ in normalized units. For large μ , $\phi_{\text{DARP}}(\mu)$ damps high-frequency modes even more strongly by driving them toward zero, which is the spectral form of the variance-reduction contribution.

Therefore, DARP’s aggregation step is mathematically equivalent to implicit Laplacian regularization with the fixed spectral filter ϕ_{DARP} , eliminating the need to tune λ explicitly. $\square$

This result gives the final method a clear interpretation: DARP is a locally adaptive implicit regularizer. Instead of introducing a global smoothness weight λ , neighbor aggregation enforces smoothness through the retrieved graph structure itself. The effective regularization varies with local degree and neighborhood geometry, adapting to the density of expert demonstrations. In spectral terms, the fixed filter $\phi_{\text{DARP}}(\mu) = 1 - \mu$ suppresses high-frequency modes more aggressively than a fixed explicit λ , as illustrated in Figure 8.

A.2 ADDITIONAL EXPERIMENTAL DETAILS

A.2.1 RETRIEVAL

Retrieval quality is central to DARP because the method’s contribution depends on exposing useful local context. Although simple Euclidean distance between current states can work, our experiments use a history-aware distance that exploits the sequential structure of demonstrations.

Let the query trajectory be $S_q = (\dots, s_{q,-1}, s_{q,0})$, where $s_{q,0}$ is the current query state s_q . To compute $d(s_q, s_i^*)$ for an expert state s_i^* , we identify its source trajectory S_j^* and timestep t , so s_i^* can be written as $s_{j,t}^*$. With lookback parameter ℓ , which specifies how many past states enter retrieval, the distance is:

$$d(s_q, s_i^*) = \sum_{n=0}^{\ell-1} \|s_{q,-n} - s_{j,t-n}^*\|$$

This distance accumulates Euclidean discrepancies over the current state and the previous $\ell - 1$ states from both the query and source trajectories, assuming valid indices. In practice, recent states should often influence retrieval more strongly, so we apply an exponential decay parameter $\gamma \geq 0$:

$$d(s_q, s_i^*) = \sum_{n=0}^{\ell-1} \|s_{q,-n} - s_{j,t-n}^*\| \cdot e^{-\gamma n}$$

Figure 9 shows how these retrieval choices affect success rate, making the sensitivity of the local-context contribution explicit.

A.2.2 CAN DARP HANDLE TASKS REQUIRING THE REPRESENTATION OF MULTI-MODAL ACTION DISTRIBUTIONS?

This experiment supports the Q2 contribution that DARP can work with richer action distributions. We evaluate on Push-T (Chi et al., 2024), where successful imitation requires representing multimodal actions (Figure 10). Here, DARP uses a Set Transformer head to predict Gaussian Mixture Model parameters. The resulting 22% improvement over BC in Table 4 shows that the difference-aware retrieval architecture can be paired with multimodal action modeling.

A.2.3 CAN DARP HANDLE DISCONTINUOUS ENVIRONMENTS WHERE NEARBY STATES MAY REQUIRE OPPOSING ACTIONS?

A reviewer-facing concern for any neighbor-based method is discontinuity: nearby states in Euclidean distance may require opposing actions. To test whether the DARP representation fails under this condition, we design a stress test using a modified version of D4RL’s Umaze environment (Figure 11).

Even in this deliberately challenging discontinuous environment, DARP achieves a 57% success rate compared to BC’s 25%. (Q3) (see Table 5) This result supports the architectural contribution of difference vectors and permutation-invariant aggregation: the model can learn to distinguish useful from misleading neighbors even when spatial proximity alone is insufficient.

A.2.4 CAN DARP RECOVER FROM BC ERROR?

This experiment directly probes the robustness contribution. We roll out a BC agent in a setting where it is known to fail; every k steps, we fork the environment and start a DARP rollout from the same state. Figure 12 shows that DARP can recover even as BC nears failure and drifts away from expert support, suggesting that difference-aware retrieval remains useful in the slightly OOD states produced by compounding BC error.

A.2.5 COMPARISON WITH CCIL

Environment	Method	Improvement over BC (%)
Hopper	CCIL	32.6%
	DARP	53.2%
Walker	CCIL	84.1%
	DARP	84.1%
Ant	CCIL	12.6%
	DARP	84.5%
HalfCheetah	CCIL	5.4%
	DARP	418.7%

Table 6: **Relative improvement over BC compared to CCIL.** On standard MuJoCo benchmarks, DARP achieves higher or equal relative gains than the explicit smoothness baseline across all tasks.

Table 6 compares DARP to CCIL, a baseline that explicitly induces smoothness. Using the reported CCIL scores, we compare percent improvement over BC. DARP is substantially stronger on three of four environments and matches CCIL on Walker, with the largest margin on HalfCheetah (418.7% vs 5.4% improvement).

A.2.6 DISTANCE METRIC SENSITIVITY

Because retrieval quality is central to DARP, we also test metric sensitivity. Although the main experiments use Euclidean distance, cosine similarity is a natural alternative in high-dimensional

Method	Success Rate (%)
BC	47
DARP (Cosine Similarity)	70
DARP (Euclidean Distance)	75

Table 7: **Distance metric sensitivity.** With R3M features on Robosuite Stack, DARP remains above BC with either Euclidean distance or cosine similarity, improving by 28% and 23%, respectively.

R3M embeddings. Table 7 shows that cosine similarity is only 5% worse than Euclidean distance while still clearly outperforming BC, suggesting that the method is not tied to a single distance metric.

A.2.7 DARP IN COMBINATION WITH DIFFUSION POLICY

Method	Stack	Threading	Peg Insertion
BC (MLP)	34	12	28
DARP (MLP)	62	38	40
Diffusion	52	44	54
DARP w/ Diffusion	82	72	66

Table 8: **DARP in combination with diffusion policy.** DARP improves both standard MLP policies and diffusion policies on Robosuite Stack, Threading, and Peg Insertion. Scores are success rates across 50 state-based trials. Because this experiment uses fewer demonstrations than Table 2, the BC and DARP numbers are not directly matched to that table.

This experiment shows that the DARP contribution is complementary to stronger action models. Diffusion policy (Chi et al., 2024) is a strong imitation-learning model class for manipulation, and Table 8 shows that a diffusion backbone can be combined with the DARP architecture. Relative to DARP with an MLP backbone, DARP with diffusion improves success by 20%, 34%, and 26% on the three tasks, for total gains over standard BC of 48%, 60%, and 38%.

Method	Training (s/epoch)	Inference (s/step)
Diffusion	5.610	0.15700
DARP (MLP)	0.559	0.00437

Table 9: **Computational efficiency context.** DARP with an MLP backbone is substantially cheaper than diffusion policy, which is $\approx 10\times$ slower in training and $\approx 36\times$ slower at inference.

Table 9 provides the corresponding efficiency context: DARP with an MLP backbone is much faster than standard diffusion, especially at inference time.

A.2.8 DARP TRAINED ON HUMAN DEMONSTRATIONS

Method	Success Rate (%)
BC	45
DARP	60

Table 10: **Human-demonstration result.** On Robosuite Stack, DARP improves over BC when both methods are trained on human demonstrations rather than RL-collected demonstrations.

Most MuJoCo and Robosuite experiments use expert demonstrations collected by an optimal reinforcement-learning policy. To check that the contribution is not limited to RL-generated demonstrations, we also train on human demonstrations for Robosuite Stack. Table 10 shows that DARP still outperforms standard BC by 15%.

A.2.9 CHOICE OF RETRIEVAL HYPERPARAMETERS

Environment	Method	Score (Mean)
Hopper	BC	507.49
	DARP	805.69
Stack	BC	0.12
	DARP	0.31

Table 11: **Validation-selected retrieval hyperparameters.** Mean scores on Hopper and Stack show that DARP maintains a performance gain over BC when training epochs and retrieval hyperparameters are selected by validation loss.

Figure 9 also makes clear that retrieval settings matter: very small neighborhoods, for example, can make DARP worse than standard BC. Table 11 shows that selecting retrieval hyperparameters and training epochs by validation loss is an effective practical strategy.

A.3 PSEUDOCODE

Algorithm 1 summarizes the complete DARP pipeline, making explicit where retrieval, difference-aware candidate prediction, and aggregation enter training.

Algorithm 1 Difference-Aware Retrieval Policies

```

1: Input: Expert demonstrations  $\mathcal{D}^* = \{(s_j^*, a_j^*)\}$ , number of neighbors  $k$ 
2: Initialize:  $f$  parameters  $\theta$ 
3: if  $g$  is parametric then
4:   Initialize:  $g$  parameters  $\psi$ 
5: end if
6: // Training Loop
7: while not converged do
8:   Sample batch of query data  $(s_q^*, a_q^*) \sim \mathcal{D}^*$ 
9:   for each query pair  $(s_q^*, a_q^*)$  in batch do
10:    // Find  $k$ -Nearest Neighbors from the entire dataset  $\mathcal{D}^*$ 
11:     $\mathcal{N}_k(s_q^*) \leftarrow \text{argmin-}k_d(s_q^*, s_j^*)$ 
12:    // Compute Neighbor-based Predictions
13:    for each neighbor index  $i \in \mathcal{N}_k(s_q^*)$  do
14:       $a'_i \leftarrow f_\theta(s_i^*, a_i^*, s_i^* - s_q^*)$ 
15:    end for
16:    // Aggregate Predictions
17:    if  $g$  is parametric then
18:       $\hat{a}_q \leftarrow g_\psi(\{a'_i\}_{i \in \mathcal{N}_k(s_q^*)})$ 
19:    else
20:       $\hat{a}_q \leftarrow g(\{a'_i\}_{i \in \mathcal{N}_k(s_q^*)})$ 
21:    end if
22:  end for
23:  // Update Parameters based on the batch loss
24:   $\mathcal{L} \leftarrow \sum_{(s_q^*, a_q^*) \in \text{batch}} \|\hat{a}_q - a_q^*\|^2$ 
25:  // Gradient descent step
26:   $\theta \leftarrow \theta - \alpha \nabla_\theta \mathcal{L}$ 
27:  if  $g$  is parametric then
28:     $\psi \leftarrow \psi - \alpha \nabla_\psi \mathcal{L}$ 
29:  end if
30: end while
31: Output: Trained parameters  $\theta$  and, if applicable,  $\psi$ 

```

A.4 RUNTIME ANALYSIS

Environment	k	Train (s/epoch)	Test (s/step)
Hopper (1,000 datapoints)	BC	0.102	0.00127
	100	0.126	0.00413
	200	0.128	0.00418
	300	0.130	0.00420
	400	0.131	0.00421
	500	0.130	0.00419
Stack (4,200 datapoints)	BC	0.431	0.00139
	100	0.556	0.00437
	250	0.539	0.00442
	500	0.559	0.00437
	750	0.576	0.00433
	1000	0.591	0.00478
	1500	0.682	0.00452
	2000	0.758	0.00451

Table 12: **Runtime comparison across environments.** Training and testing speeds (seconds) for BC and DARP with varying k on Hopper and Stack, showing the cost of adding retrieval context.

Table 12 quantifies the computational cost of adding retrieval context to BC. The overhead grows sub-linearly as k increases, and inference remains tractable for real-time robotic use. For example, on Robosuite Stack with $k = 500$, DARP takes approximately 0.00437 seconds per step on our hardware, corresponding to a control frequency above 230 Hz.

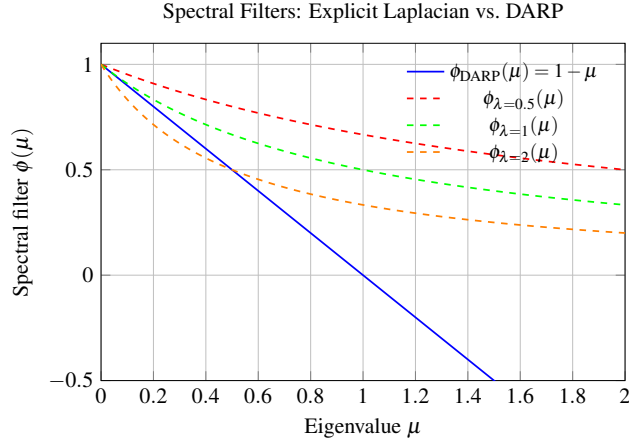


Figure 8: DARP implements a parameter-free low-pass filter that is sharper than the shown explicit Laplacian filters.

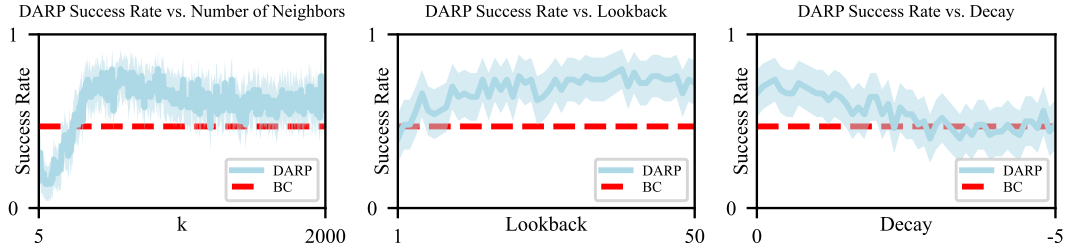


Figure 9: **Retrieval hyperparameters control how much local context supports the DARP contribution.** Left: very small neighborhoods perform poorly, performance peaks around 500 retrieved neighbors, and very large neighborhoods plateau slightly above BC. Center: including more history generally helps, while little or no history can underperform BC. Right: excessive decay on older observations behaves similarly to little lookback and can hurt performance. Success rate is measured over 50 Robosuite Stack trials with 95% confidence intervals.

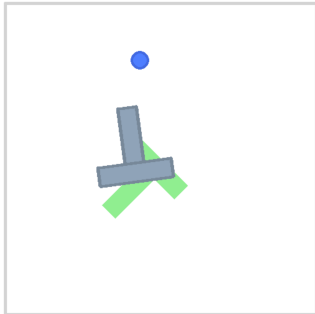


Figure 10: **Push-T environment for Q2.** The task requires controlling the blue circle to push the T-shaped block, creating a multimodal action-distribution test for DARP.

Method	Score
BC	48 ± 8
DARP	70 ± 8

Table 4: **Push-T result.** Averaged over 100 trials, DARP outperforms BC on the multimodal action task.

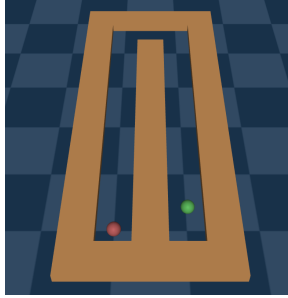


Figure 11: **Long maze stress test.** The task moves a force-actuated ball from the green start to the red destination and tests DARP under discontinuous local action structure.

Method	Succ. (%)
BC	25
DARP	57

Table 5: **Long maze result.** Averaged over 100 trials, DARP substantially outperforms BC in the discontinuity stress test.

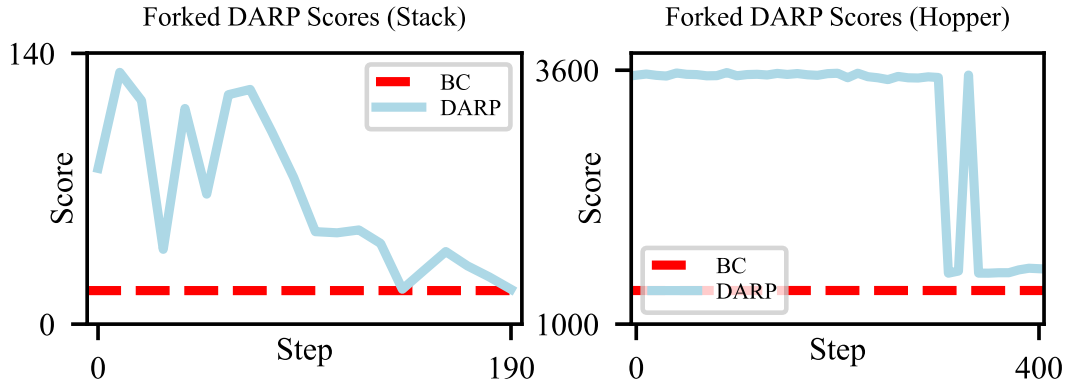


Figure 12: **Recovery from BC drift.** On Robosuite Stack and MuJoCo Hopper, we roll out a failing BC agent and fork the environment every k steps ($k = 10$) to start a DARP rollout. DARP retains high scores deep into the failing BC trajectory, with recovery limited only about halfway through Stack and about 80% through Hopper.